数位统计DP

数位统计DP用于数字的数位统计。

一个数字的数位有个位、十位、百位，等等，如果题目和数位统计有关，那么可以用DP思想，把低位的统计结果记录下来，在高位计算时直接使用低位的结果，从而提高效率。

数位统计相关的题目，基本内容是处理“前导0”和“数位限制”。

1.1 前导0:

例:1~99内有多少个0、1、2、3、4...?

转化为统计00~99，即:

00、01、02、03、04、05、06、07、08、09
10、11、12、13、14、15、16、17、18、19
20、21、22、23、24、25、26、27、28、29
.....
90、91、92、93、94、95、96、97、98、99

数字一共出现了$2x10^2$=200个，平均每个数出现了20次。

- 1、2、...、9:都出现了20次
- 0特殊，因为00~09中的0不能算，所以0出现了20-10-1=9次。
- 称00、01、02、...中的0为“前导0”。
- 类似的有001、010、0100、...的前导0

1.2 数位限制

例:1~23内有多少个0、1、2、3、4...?

转化为00~23:

00,01,02,03,04,05,06,07,08,09:

最高位0出现10次，个位的其他数(0~9)各出现1次

10, 11、12, 13, 14, 15, 16, 17, 18, 19

最高位1出现10次，个位的其他数(0~9)各出现1次

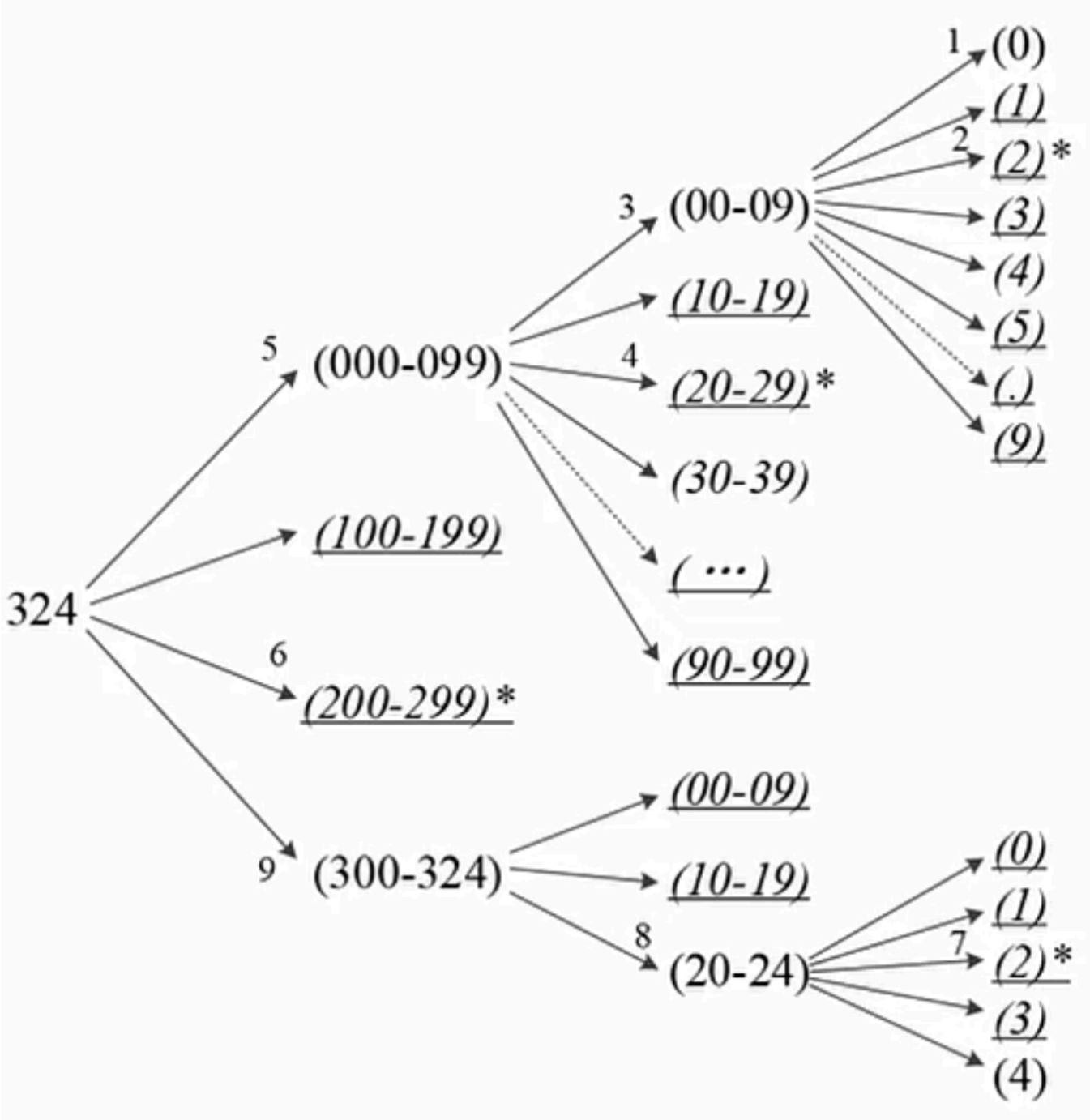
20,21,22,23

最高位是2，称2为“数位限制”，此时需要特别判断。

下面用一道例题详解数位统计DP的建模和两种实现：递推、记忆化搜索。

1.3 例题 数字计数（洛谷P2602）

- 问题:给定正整数a和b，求在[a，b]中的所有整数中，每个数码(digit)各出现了多少次。



以统计[0, 324]内数位2出现了多少次为例，搜索过程如图所示，其中，下划线的数字区间是前面已经计算过的，记录在状态dp[]中，不用再重算，符号*表示区间中有数位2，需要特殊处理。

[0,324]分解为4个区间:000 ~ 099、100 ~ 199、200 ~ 299、300 ~ 324

其中000 ~ 099、100 ~ 199、200 ~ 299能够沿用00~99的计算结果。

300~324，特判

2.1 数位统计DP的递推实现

定义状态dp[]: i位数的每种数字有多少个。

一位数0~9，每种数字有dp[1]=1个。

二位数00~99，每种数字有dp[2]=20个。

三位数000~999，每种数字有dp[3]=300个;

四位数0000~9999，dp[4]=4000;...等等。

- dp[i]=dp[i-1]X10+10^{i-1},这是从递推的角度分析得到的。以数字2为例，计算dp[2]时，在个位上出现了dp[i-1]X10=dp[1]X10=10次，即2，12，22，...，92；在十位上出现了$10^{i-1}=10^{2-1}$=10次，即20，21，22，...，29。计算dp[3]时，2在个位和十位上出现了dp[2]X10=200次，在百位上出现了 10^{3-1}=100次。
- 按排列组合的思路，可得 dp[i]=(iX10^i)/10。因为从i个0递增到i个9，所有的字符共出现iX10^i次，0 ~ 9每个数字出现了iX10^i/10次。从i个0递增到i个9，所有的字符共出现了iX10^i次，0~9这10种字符每种出现了(iX10^i)/10次。

```
void solve(ll x, ll *cnt){
    int len = 0;  //数字x有多少位
    while(x){     //分解x，num[i]是x的第i位数字
        num[++len] = x%10;
        x=x/10;
    }
    for(int i=len;i>=1;i--){                            //从高到低处理x的每一位
        for(int j=0;j<=9;j++)      cnt[j] += dp[i-1]*num[i]; //cnt[j] ,统计数字j出现了多少次
        for(int j=0;j<num[i];j++)  cnt[j] += ten[i-1];  //特判最高位比num[i]小的数字
        ll num2 = 0;
        for(int j=i-1;j>=1;j--)    num2 = num2*10+num[j];
        cnt[num[i]] += num2+1;                          //特判最高位的数字num[i]
        cnt[0] -= ten[i-1];                             //特判前导0
    }
}
```

分析代码的复杂度，solve()函数有两层for循环，只循环了10Xlen次。

2.2 数位统计DP的记忆化搜索实现

回顾记忆化搜索，其思路就是在递归程序dfs()中，搜索所有可能的情况，遇到已经算过的记录在dp中的结果，就直接使用，不再重复计算。

定义状态dp[][]

dp[pos][sum]:表示最后pos位范围是[pos个0,pos个9]，前面2的个数是sum个时，数字2的总个数。例如:

dp[1][0]=1,表示00-09、10~19、30-39....区间内2的个数有1个。

dp[1][1]=11，表示20~29区间内2的个数有11个。

dp[1][2]=21，表示220~229区间内的2有21个。

dp[2][1]=120，表示200~299内的2有120个。

dp[2][2]=220，2200~2299内的2有220个。

```
ll dfs(int pos,int sum,bool lead, bool limit){    //pos：当前处理到第pos位
    ll ans = 0;
    if(pos == 0) return sum;    //递归到0位数，结束，返回
    if(!lead && !limit && dp[pos][sum]!=-1)  return dp[pos][sum];  //记忆化搜索
    int up = (limit ? num[pos] : 9);    //这一位的最大值，例如324的第3位是up = 3
    for(int i=0;i<=up;i++){                                          //下面以now=2为例：
        if(i==0 && lead)  ans += dfs(pos -1, sum,  true, limit&&i==up); // 计算000~099
        else if(i == now) ans += dfs(pos -1, sum+1,false,limit&&i==up); // 计算200~299
        else if(i != now) ans += dfs(pos -1, sum,  false,limit&&i==up); // 计算100~199
    }
    if(!lead && !limit) dp[pos][sum] = ans;       //状态记录：无前导0
    return ans;
}

```